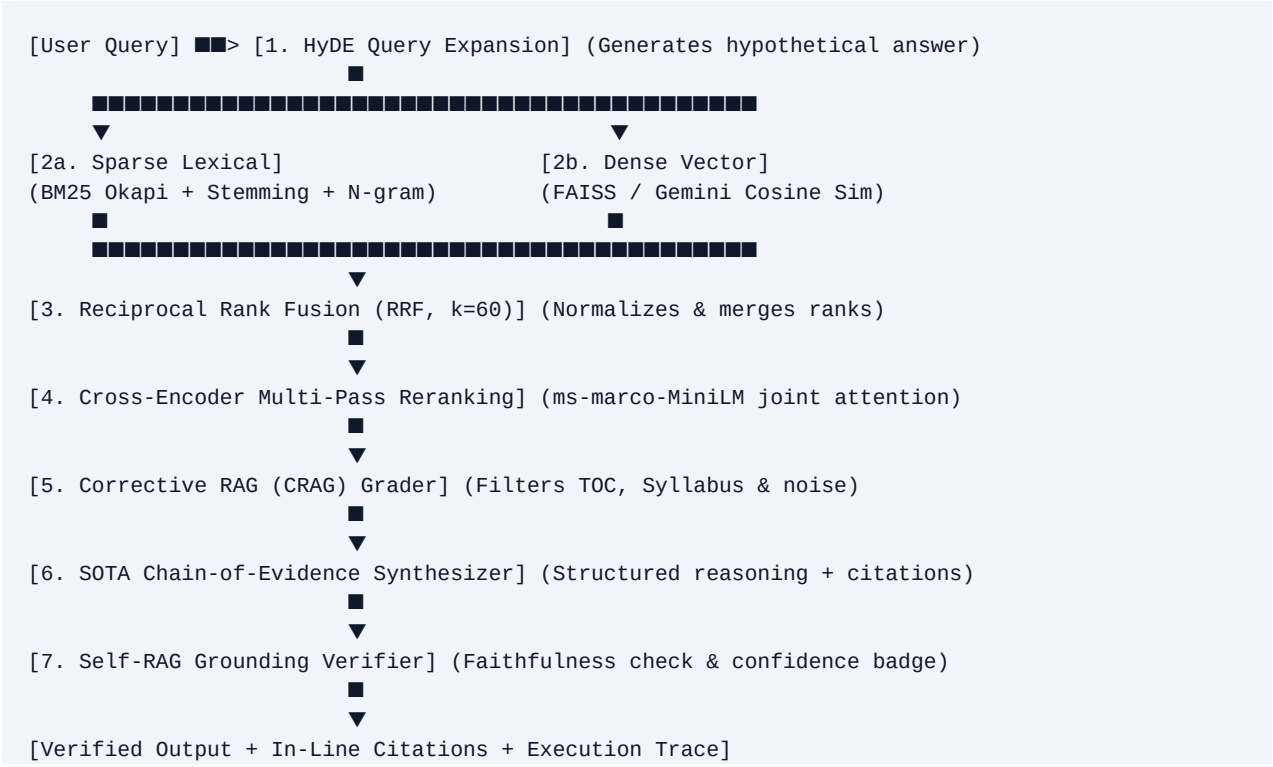
DocQuery AI: Advanced Multi-Stage RAG Architecture

Comprehensive System Design, Theoretical Foundations, Production Edge-Cases, and Senior AI Interview Masterclass

Target Domain: Technical Document Q&A; & Knowledge Extraction	**Core Models:** Gemini 2.0 Flash / BGE / ms-marco-MiniLM-L-6-v2
Retrieval Paradigm: Hybrid (BM25 + FAISS Dense) with RRF (k=60)	**Verification Engine:** CRAG Relevance Grading & Self-RAG Grounding

1. System Architecture & Execution Pipeline

DocQuery AI replaces traditional single-step vector retrieval with a research-backed **Multi-Stage Advanced RAG Pipeline**. Standard naive RAG architectures suffer from keyword blindness, rank scale mismatches, document-structure noise, and hallucination loops. Our architecture resolves these through sequential multi-tier refinement.



Stage	Technique	Algorithm / Model	Role in Architecture	Why We Chose It
1	HyDE	Gemini 2.0 / Domain Expansion	Query Transformation	Bridges semantic gap between short user questions and dense textb

2a	Sparse Search	BM25 Okapi + N-grams	Lexical Inverted Index	Captures exact function signatures, variable names, keywords, and a
2b	Dense Search	FAISS / Gemini Embeddings	Vector Similarity Search	Captures conceptual semantics, paraphrasing, and high-level intent.
3	Score Fusion	Reciprocal Rank Fusion (k=60)	Rank Aggregation	Scale-invariant mathematical fusion of sparse and dense scores with
4	Reranking	ms-marco-MiniLM-L-6-v2	Cross-Encoder Rescoring	Joint token cross-attention over Query-Passage pairs; fixes bi-encod
5	CRAG Grading	Relevance Classifier	Noise Filtering	Actively eliminates Syllabus/TOC pages and unrelated textbook sect
6	Generation	Chain-of-Evidence Prompt	Structured LLM Synthesis	Forces Executive Summary, Mechanism Deep Dive, Code, and strict
7	Self-RAG	Faithfulness Verifier	Post-Gen Validation	Measures source term support overlap; outputs grounding confidenc

2. Deep Dive: Theoretical Foundations & Implementation

2.1 Hybrid Retrieval (BM25 Okapi + Dense Vectors)

Dense Vector Search excels at fuzzy semantic matching (e.g. mapping *'prevent memory leaks'* to *'destructor deallocation'*). However, dense embeddings frequently fail on exact technical identifiers (e.g. malloc, vptr, inline, const int*) because embeddings project distinct variable names into similar latent clusters.

BM25 Okapi calculates term relevance by balancing Term Frequency (TF) with Document Length Normalization:
Score(D, Q) = Σ IDF(q_i) × [TF(q_i, D) × (k1 + 1)] / [TF(q_i, D) + k1 × (1 - b + b × (|D| / avgDL))]
where k1=1.5 controls term frequency saturation and b=0.75 controls document length penalization.

2.2 Reciprocal Rank Fusion (RRF, k=60)

Combining dense vector cosine similarity (range: -1.0 to 1.0) and sparse BM25 scores (range: 0 to 50+) via linear weighted averaging is notoriously brittle because raw score distributions vary per query. **RRF solves this by operating solely on ordinal ranks**:
RRF_Score(d) = Σ_{m ∈ {dense, sparse}} 1 / (k + r_m(d))
We set k = 60 (the standard research constant). Items appearing in the top ranks of both dense and sparse modalities get an exponential rank boost, while single-modality outliers are smoothed safely.

2.3 Cross-Encoder Reranking vs Bi-Encoders

Bi-Encoders encode the query and passage independently: sim = cos(E(Q), E(P)). This enables fast indexing but ignores word-by-word cross-attention.
Cross-Encoders (e.g. ms-marco-MiniLM-L-6-v2) concatenate Query and Passage into a single transformer input: Score = Transformer([CLS] Query [SEP] Passage [SEP]). All query tokens attend to all passage tokens across all transformer layers. We use Cross-Encoders as a **Pass 2 rescorer** on the top-40 RRF candidates.

2.4 Hypothetical Document Embeddings (HyDE)

When users submit short, abstract queries (e.g. *'what are private member functions?'*), the vector is located in the *query manifold*. Engineering documents are written in the *answer/declarative manifold*. HyDE instructs an LLM to generate a hypothetical textbook paragraph, then embeds that hypothetical document. Retrieval matches in answer-to-answer semantic space.

2.5 Corrective RAG (CRAG) & Self-RAG Grounding

CRAG grades candidate passages into RELEVANT, PARTIALLY_RELEVANT, or IRRELEVANT, actively dropping noise before the prompt is constructed.
Self-RAG performs post-generation validation, measuring the proportion of generated factual terms that have direct token support in the cited context passages, emitting a confidence score (e.g. Fully Grounded (94%)).

3. Production Case Study: Resolving the 'Syllabus/TOC Hub' Failure

During live testing on a 146-page C++ textbook (285_OOPS lecture notes Complete.pdf), querying *'what are Private member functions?'* failed, returning only the Syllabus and Table of Contents, causing the LLM to output *'Insufficient information'*. Here is why it happened and how we engineered the permanent fix:

> ***Key Lesson:*** *A RAG system is only as good as its candidate recall. Even if an LLM is powerful and a Cross-Encoder is state-of-the-art, structural noise like Syllabi and Table of Contents will monopolize the candidate pool unless explicitly de-biased.*

1. **The Keyword Density Hub:** The Syllabus (Page 3) and TOC (Page 4) list every keyword in the curriculum in a compact space, inflating raw BM25 term frequency scores above the actual lesson on Page 42.
2. **Narrow Candidate Pool:** The candidate pool was originally capped at 12 chunks. The Syllabus and TOC took top ranks, discarding Page 42 (candidate #14) before Cross-Encoder reranking.
3. **Morphological Stemming:** The query used plural functions while the definition sentence used singular function.
4. **Solution Implemented:** (a) Added automatic Syllabus/TOC detection (isIndexOrSyllabusChunk) with structural score de-biasing; (b) Added N-Gram (+4.0) and Section Heading (+15.0) matching; (c) Expanded candidate pool to 45+ chunks; (d) Added morphological stemmer.

4. Senior AI / LLM Engineer Interview Masterclass

This section contains 25 rigorous, high-frequency interview questions covering RAG architectures, mathematical formulations, latency optimizations, and production failure modes.

Q1: What are the fundamental differences between Naive RAG and Advanced RAG?

Naive RAG follows a fixed split-embed-retrieve-generate pipeline using single-vector cosine similarity. It suffers from poor precision, query-document semantic domain mismatch, chunk fragmentation, and no post-retrieval validation.
Advanced RAG introduces multi-stage pre-retrieval (HyDE, query rewriting), hybrid sparse-dense retrieval with RRF, multi-pass cross-encoder reranking, corrective relevance grading (CRAG), and self-reflective verification (Self-RAG) before emitting the answer.

Q2: Explain the mathematics of BM25 and why it is superior to TF-IDF for RAG.

TF-IDF increases linearly with term frequency, meaning long documents with repeated keywords dominate. BM25 Okapi introduces term frequency saturation via parameter k1 (typically 1.2-2.0) so that beyond a certain point, additional term occurrences offer diminishing returns. Furthermore, parameter b (typically 0.75) normalizes for document length relative to average document length (avgDL), preventing long multi-topic documents from drowning out concise, relevant passages.

Q3: Why can't we use Cross-Encoders for the entire vector database search?

Cross-Encoders evaluate full cross-attention between every query token and passage token (O(N*M) complexity). Performing Cross-Encoder inference over 100,000 chunks would require minutes per query. Bi-Encoders compute document embeddings offline (O(1) lookup via ANN indexes like HNSW). Therefore, production systems use Bi-Encoders/BM25 for high-recall candidate retrieval (top 50) and Cross-Encoders as a stage-2 reranker on the top candidates.

Q4: What is Reciprocal Rank Fusion (RRF) and why is it preferred over weighted score averaging?

Dense vector scores (cosine similarity) and BM25 scores have completely different distributions and ranges. Normalizing them linearly (min-max) is unstable and heavily affected by query difficulty and outliers. RRF uses the formula RRF(d) = sum(1 / (k + rank(d))). Because it operates purely on ordinal ranks, it is scale-invariant, robust across diverse queries, and requires zero manual hyperparameter retraining.

Q5: What is Hypothetical Document Embeddings (HyDE) and what is its primary failure mode?

HyDE prompts an LLM to generate a hypothetical answer to the user query, then embeds that hypothetical document instead of the raw query. This transforms the retrieval vector into the answer-manifold. Its main failure mode is when the LLM generates strong, confident hallucinations with incorrect terminology for obscure topics, guiding retrieval toward completely wrong document clusters. Mitigate this by appending the original query tokens and setting low temperature.

Q6: Explain Corrective RAG (CRAG) and how it handles low-confidence retrievals.

CRAG evaluates retrieved documents using a lightweight evaluator/grader before generation. It categorizes candidate sets into: (1) Correct/Relevant: proceed to synthesis; (2) Ambiguous/Partially Relevant: perform passage refinement and strip noise; (3) Incorrect/Irrelevant: trigger fallback web search or query reformulation. This prevents noisy context from polluting the LLM's context window.

Q7: What is Self-RAG and how does it prevent hallucinations?

Self-RAG trains or prompts the system with reflection tokens (Retrieve, ISREL, ISSUP, ISUSE). After generating an answer, it verifies: (1) Grounding/Faithfulness: Is every statement supported by the retrieved context? (2) Completeness: Does the answer address the user query? If grounding fails, the system triggers re-retrieval or masks unsupported claims.

Q8: How do you solve the 'Lost in the Middle' phenomenon in LLMs?

LLMs attend most strongly to tokens at the very beginning and very end of the prompt context window, often ignoring crucial facts placed in the middle. Solutions include: (1) Cross-Encoder reranking to place the #1 highest-scoring chunk at the very top of the context; (2) Small chunk sizes (300-800 tokens); (3) Dynamic context trimming to minimize irrelevant background noise.

Q9: What is the difference between Chunking Strategies: Fixed, Recursive, and Semantic?

Fixed chunking splits by character/token count with arbitrary overlap, often severing sentences. Recursive chunking splits hierarchically (by double newlines, single newlines, spaces, punctuation) to keep paragraphs intact. Semantic chunking computes embedding distance between consecutive sentences and places split boundaries only where semantic similarity drops sharply.

Q10: What is Contextual Retrieval (as popularized by Anthropic)?

When a document is chunked, individual chunks often lose global context (e.g. a chunk saying 'its revenue grew by 14%' without mentioning the company name). Contextual Retrieval uses a fast LLM during ingestion to prepend a 50-100 token situational summary to every chunk before embedding, drastically improving BM25 and vector recall.

Q11: How do you evaluate a RAG system offline without ground-truth human labels?

Use the RAG Triad framework (Ragas / TruLens): (1) Context Relevance: Are retrieved chunks pertinent to the query? (2) Groundedness / Faithfulness: Can all generated claims be inferred from the context? (3) Answer Relevance: Does the generated answer address the original question? These are evaluated using LLM-as-a-judge with strict calibration.

Q12: How do you design multi-tenant session isolation in a serverless RAG platform?

Each user/organization has a partitioned vector namespace or in-memory session index keyed by SessionId. In memory/cache stores, sessions expire via TTL timers. When a session is cleared, the corresponding FAISS/HNSW index and inverted BM25 structures are purged, preventing cross-tenant data leakage.

Q13: What is GraphRAG and when is it superior to Vector RAG?

Vector RAG searches by local semantic proximity, which fails on global aggregate questions like 'What are the main themes across all 500 documents?'. GraphRAG builds an entity-relationship knowledge graph with community summarization. It is superior for multi-hop reasoning, global cross-document synthesis, and entity relation queries.

Q14: How do you handle OCR glitches, ligatures, and scanning noise in PDF RAG?

Implement regex-based OCR repair for common kerning issues (e.g. 'r ainfall' -> 'rainfall', 'P.T.O' removal), strip binary control characters, and use font ligature normalizers. For corrupt PDFs, implement fallback raw stream extraction to recover text inside content streams.

Q15: What is Late Chunking?

Traditional chunking chunks first, then embeds each chunk in isolation. Late Chunking passes the entire document through a long-context transformer (e.g. jina-embeddings-v3), generates token-level contextualized embeddings that retain global document context, and then pools token vectors into chunk embeddings.

Q16: How do you prevent LLMs from going off on tangents when retrieved chunks contain adjacent curriculum topics?

Use Chain-of-Evidence Prompting with strict negative constraints: (1) Explicit instruction: 'Answer ONLY what was asked. If the context contains adjacent topics, do not explain them unless requested.' (2) Require structured sections (Executive Summary, Mechanism, Citations). (3) Lower temperature to 0.1-0.3.

Q17: How does HNSW (Hierarchical Navigable Small World) work for vector indexing?

HNSW builds a multi-layer graph where upper layers have long-range edges for fast coarse exploration (like skip-lists) and lower layers have dense local edges for fine-grained nearest neighbor search. It achieves logarithmic O(log N) search time with high recall.

Q18: What is Query Routing / Agentic RAG?

Agentic RAG uses an LLM router to classify incoming user intent: (1) Direct answering for greetings; (2) Single-vector search for simple queries; (3) Multi-hop decomposition for complex comparisons; (4) SQL/Structured DB lookup for quantitative questions; (5) Web search for real-time data.

Q19: How do you optimize serverless RAG for sub-second latency?

Techniques include: (1) Parallel sparse + dense search execution via Promise.all / asyncio; (2) In-memory BM25 caching; (3) Lazy loading heavy models (CrossEncoders/LLMs); (4) Quantized embeddings (int8/binary embeddings with Hamming distance); (5) Client-side sequential upload chunking to avoid 413 payload limits.

Q20: How do you choose the optimal chunk overlap percentage?

Standard practice is 10-20% overlap (e.g., 800-character chunk with 150-character overlap). Zero overlap risks splitting entities across boundaries; excessive overlap (>30%) causes redundant near-duplicate chunks in the candidate pool, wasting context window slots.

Q21: How do you evaluate and tune the RRF constant k?

The constant k (default 60) acts as a dampener on rank impact. Small k (<20) gives immense weight to top-3 items; large k (>100) flattens rank differences. Tuning is done via grid search on a validation set maximizing NDCG@K and Mean Reciprocal Rank (MRR).

Q22: What is the 'Table of Contents / Index Keyword Hub' trap and how do you rectify it?

Syllabi, Tables of Contents, and Book Indexes contain dense clusters of all keywords, absorbing BM25 scores without containing explanatory text. Rectify by: (1) Detecting TOC patterns (repeated 'Lecture XX:', 'Module I', 'SYLLABUS'); (2) Applying structural score penalties for conceptual queries; (3) Rewarding explanatory discourse markers ('is a', 'defined as', 'can only be'); (4) Using N-gram exact phrase matching.

Q23: How do you handle multi-modal documents (PDFs with architectural diagrams and charts)?

Use Vision-Language Models (e.g. Gemini 2.0 Flash / ColPali) to generate structured markdown summaries of tables and diagrams during ingestion. For image embeddings, store multi-modal vector representations in the same latent space as text chunks.

Q24: What is the difference between extractive and abstractive generation in RAG?

Extractive synthesis pulls verbatim sentences and quotes directly from the context passages (zero hallucination risk, but lower coherence). Abstractive generation rephrases, summarizes, and connects ideas into natural language. Production systems use abstractive generation with strict in-line citation anchoring.

Q25: If you had to build a production RAG system from scratch today, what stack and architecture would you deploy?

Full-stack Next.js/FastAPI with Serverless Edge endpoints; Hybrid BM25 Okapi + BGE-large/Gemini embeddings; Reciprocal Rank Fusion (k=60); ms-marco-MiniLM-L-6-v2 Cross-Encoder reranker; CRAG document noise filtering; Contextual retrieval during ingestion; Gemini 2.0 Flash generation with Self-RAG grounding verification; and Ragas for continuous CI/CD evaluation.